Software Requirements Specification (SRS)

IEEE 830-style specification

System: TFS HRMS (shown to users as "TFS Ops Tracker")

Version 1.0 · 23 October 2026 · Prepared from the source code (backend-api, qc-backend, hrms-frontend-live)

Document control

Item	Value
Document	HRMS Software Requirements Specification
Version	1.0 (as-built, reverse-engineered from code)
Date	23 October 2026
Prepared by	Ummehabiba Siddiquie (Development)
Status	Draft for review – items marked 'To be confirmed' need business sign-off

Contents

- 1. Introduction
- 2. Overall description
- 3. User roles and permissions
- 4. Functional requirements
- 5. Non-functional requirements
- 6. Data model
- 7. External interfaces and integrations
- 8. Business rules
- 9. Assumptions, constraints and dependencies
- 10. Known gaps and issues found in the code
- 11. Requirements traceability matrix
- Appendix A. Workflow diagrams

1. Introduction

1.1 Purpose

This document specifies the functional and non-functional requirements of the TFS HRMS ("TFS Ops Tracker"), the in-house operations system used to track agent production, quality checks (QC), rosters, QA hours, monthly goals and KRA. It is written from the current source code so it describes the system as built; items that could not be confirmed from code are marked 'To be confirmed'. Audience: management, HR/operations leads, QA leads, developers and testers.

1.2 Scope

In scope: the React website (hrms-frontend-live), the Python Flask API (backend-api), the Node.js QC API (qc-backend), the shared MySQL database, scheduled jobs and report emails. Out of scope: payroll, attendance devices, leave balances/accrual, recruitment and any HR function not present in the code.

1.3 Definitions, acronyms and abbreviations

Term	Meaning
AFD	Named list of QC error codes, categories and penalty points used to score a file
Agent	Production employee who logs work in the Tracker
QA	Quality analyst who performs QC on agents' files
QC	Quality check of a sample of rows in an agent's file
Regular / Correction / Rework	QC results: pass (100), fix marked rows (98–<100), fix whole file (<98)
Tracker entry	One record of production (project, task, shift, count, file)
Tenure	Productivity factor per user (e.g. 1, 0.75, 0.5) used for targets
Billable hours	Production ÷ (task target × tenure)
Assigned hours	Hours expected on a day from the roster (9 × tenure for a full day)
Roster	Per-person monthly plan of working days, week offs, leave, holidays and shifts
Change request	A roster change (day, week-off swap, leave) awaiting Admin approval
Week lock	Admin lock on a Monday–Sunday week that blocks all roster changes
KRA	Key Result Area score combining productivity, quality, adherence, reporting and timeliness
EOD report	End-of-day consolidated Excel for one task and day
SOP	Standard operating procedure document/link in the SOP library
TBC	To be confirmed

1.4 References

- Source code: backend-api (Flask), qc-backend (Node/TypeScript), hrms-frontend-live (React)
- Existing project docs: docs/ROLE_BASED_ACCESS.md, docs/ROLE_BASED_WORKFLOW.md, docs/ADMIN_SUPER_ADMIN_ACCESS.md, handover documents 01–08
- HRMS Detailed System Workflow (companion PDF)

2. Overall description

2.1 Product perspective

A three-tier web system. The browser talks to two APIs which share one MySQL 8 database; files are stored on Cloudinary and emails are sent over SMTP.

Component	Technology	Responsibility
Website	React 19, Vite 7, Tailwind 4, React Router 7, axios	All screens; role-based navigation; QC form scoring in the browser
Flask API	Python 3, Flask 3, mysql-connector, pandas, openpyxl, APScheduler, Cloudinary SDK, bcrypt/cryptography	Auth, users, projects, tasks, tracker, roster, holidays, reports, KRA, QA hours, SOP, report emails, 08:00 job (26 blueprints, ~126 endpoints)

Node QC API	Node, TypeScript, Express 5, mysql2, ExcelJS, xlsx, nodemailer, Cloudinary, Vercel AI SDK (Gemini), zod	QC sample generation, QC saves (regular/correction/rework), annotated files, agent re-upload, duplicate check, AI evaluation, Gemini keys
Database	MySQL 8 (live: mytfs on Hostinger VPS)	~35 tables; mostly soft delete; only 2 real foreign keys
Hosting	Nginx reverse proxy on a VPS; /python → Flask :5000, /api/v1 → Node :8000; pm2	Vercel used for test builds only

2.2 Product functions (summary)

- Login, password reset and role-based access
- User, role, team, reporting-line and permission management
- Project, task, project category and AFD (QC error list) management
- Agent daily production tracker with file upload, billable hours and duplicate check
- QC workflow: sampling, error marking, automatic scoring, correction and rework cycles, emails, audits
- QA hours tracking (automatic and manual)
- Roster generation, change requests, approval, week/month locks, holiday master, Excel upload, roster emails
- Reports: overview, billable, tracker report, QA report, user/project monthly goals, QC report overview, QA agent audit, KRA, Task EOD report
- AI evaluation of agent files (Google Gemini)
- SOP library; report email recipient management; scheduled jobs

2.3 Operating environment

- Modern desktop browser (Chrome/Edge); sessionStorage must be enabled
- Linux VPS with Python 3.11+, Node 18+, MySQL 8, Nginx, pm2; India Standard Time (IST) used for business dates
- Upload limit 10 MB per request at Nginx

2.4 Design and implementation constraints

- No authentication token: the browser sends logged_in_user_id with each request and the APIs look up the role in the database
- CORS open to all origins on both APIs
- New MySQL connection per request (no pool)
- Some tables/columns are created by code on first use rather than by migrations

3. User roles and permissions

role_id	Role	Lands on	Data scope	Key rights
1	Super Admin	Dashboard	Whole organisation	Everything, incl. User Permission flags and Report emails
2	Admin	Dashboard	Whole organisation	Everything except Report emails; approves roster, locks weeks, holiday master
3	Project Manager	Dashboard	Whole organisation	Users, projects, roster edit/submit, reports, QC overview, audit; cannot approve roster
4	Assistant Manager	Dashboard	Assigned users (asst_manager_id)	Projects, roster edit/submit (team-locked), monthly goals; no user management
7	Assistant Team Leader	Dashboard	Own team	Read-only except roster edit/submit
5	QA	Dashboard	Assigned agents (qa_id) + self	QC form, Agent Files, My Report (QA hours), KRA notes
6	Agent	/agent (Tracker)	Self only	Tracker, own reports & QC results, KRA, My Roster, AI Evaluation

Additional permission flags per user (user_permission): User Creation and Project Creation, set by Super Admin. Salary/incentive visibility is limited to Admin, Super Admin, Project Manager and FINANCE_HR users (frontend rule – backend enforcement TBC).

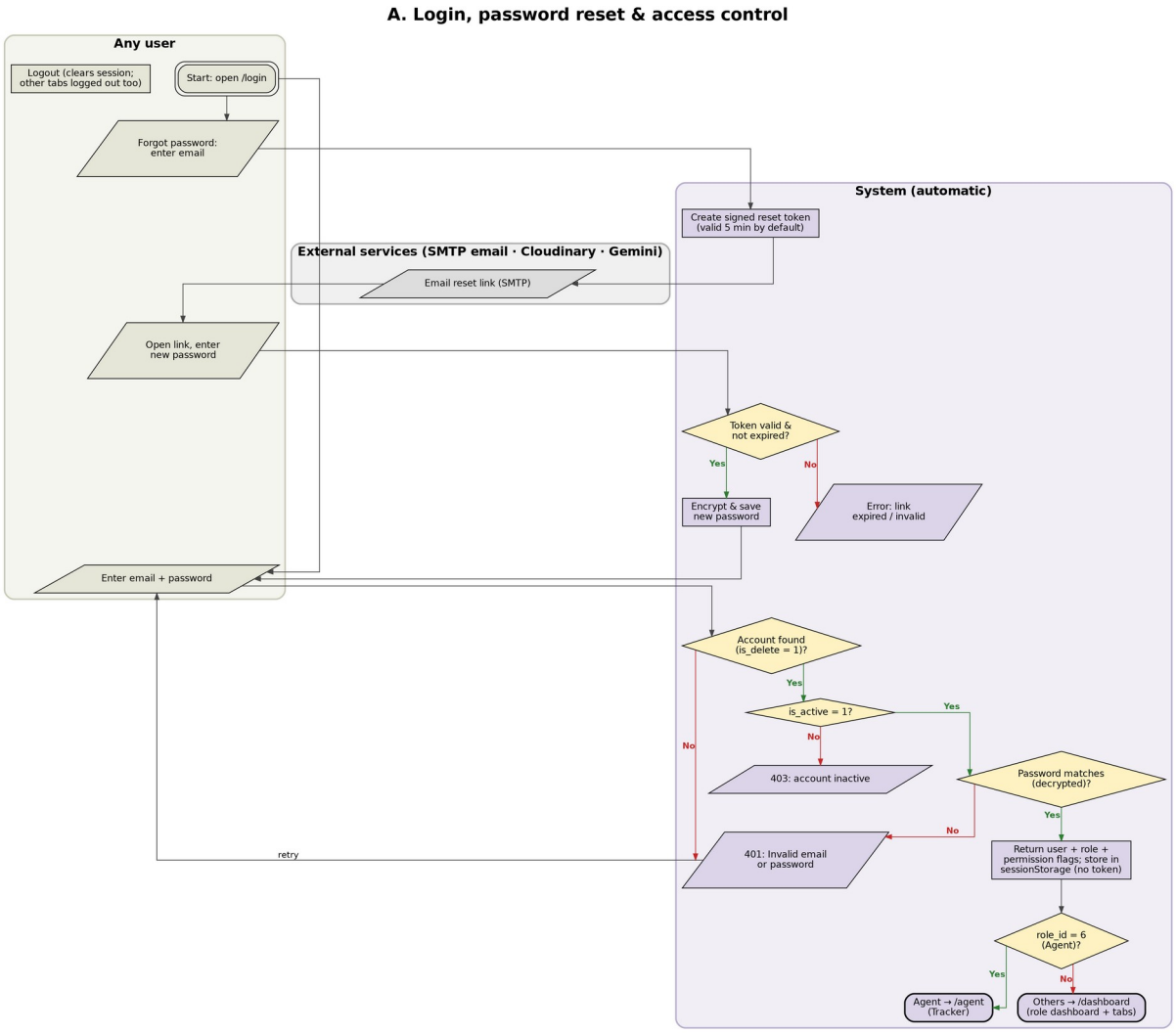


Figure 1 – Login & access

4. Functional requirements

4.1 Authentication & access (AUTH)

ID	Requirement	Source in code
FR-01	The system shall authenticate users by email and password and reject unknown, deleted (is_delete=0) or wrong-password logins with 'Invalid email or password'.	routes/auth.py
FR-02	The system shall block login for inactive accounts (is_active ≠ 1) with HTTP 403.	utils/user_status.py
FR-03	On success the system shall return the user profile, role and permission flags (never the password) and the website shall keep it in sessionStorage.	auth.py, AuthContext.jsx
FR-04	The website shall route Agents to /agent and all other roles to /dashboard, and redirect users who open a page not allowed for their role to their home page.	routes/AppRoutes.jsx, paths.js
FR-05	A new login or logout in one browser tab shall log out other tabs of the same browser.	AuthContext.jsx
FR-06	Users shall be able to request a password-reset link by email; the link shall be signed and expire after RESET_TOKEN_TTL_SECONDS (default 300 s).	routes/password_reset.py
FR-07	The system shall verify the reset token before accepting a new password and store	password_reset.py

	the password encrypted.	

4.2 Users & permissions (USR)

ID	Requirement	Source in code
FR-08	Only Super Admin, Admin, Project Manager or users with the User Creation flag (never Team Leaders) shall create users.	auth.py (registration)
FR-09	User creation shall require name, email, password and role, validate email/phone/password format and accept an optional profile picture.	auth.py, utils/validators.py
FR-10	Each user shall have role, team, designation, tenure, joining date and reporting lines (project manager, assistant manager, team leader, QA) stored as ID lists.	tfs_user
FR-11	Authorised users shall edit users, change passwords and deactivate users with a leaving date (deactivated_at).	routes/user.py
FR-12	Deactivated users shall not log in but may add/edit tracker entries up to and including their leaving date.	utils/user_status.py
FR-13	User delete shall be a soft delete (is_delete = 0).	routes/user.py
FR-14	Super Admin shall set per-user User Creation and Project Creation flags.	routes/user_permission.py

4.3 Projects, tasks & AFD (PRJ)

ID	Requirement	Source in code
FR-15	Authorised users shall create, edit and delete AFD masters with error categories, error names and points.	routes/afd_master.py, qc_afd.py
FR-16	Project categories shall be created and linked to exactly one AFD.	routes/project_category.py
FR-17	Projects shall store code, category, manager, teams, QA list, files (Cloudinary), SOP links, AI-evaluation flag and duplicate-check flag.	routes/project.py
FR-18	Tasks shall store target per hour, QC %, important columns and exactly one QA target mode (object ranges / minutes per file / minutes per record).	routes/task.py
FR-19	Only users permitted to manage projects (roles 1–4 or Project Creation flag, not Team Leader) shall create or edit projects.	project.py, AuthContext.jsx

4.4 Daily tracker (TRK)

ID	Requirement	Source in code
FR-20	Agents shall add tracker entries with project, task, shift (DAY/NIGHT), production count, optional file and note.	routes/tracker.py
FR-21	The system shall reject tracker entries for future dates.	tracker.py
FR-22	A NIGHT-shift entry created before 09:00 shall be dated to the previous day.	tracker.py
FR-23	The system shall compute tenure_target = task_target × tenure and billable_hours = production ÷ tenure_target and store both with the entry.	tracker.py
FR-24	When a project has duplicate check enabled, uploaded Excel rows shall be hashed on the task's important columns and a file with an already-uploaded row shall be rejected.	qc-backend tracker-process.controller.ts
FR-25	Agents shall delete only same-day entries (night entries until 09:00 next day); managers may delete any entry. Delete is soft and removes the Cloudinary file.	tracker.py
FR-26	Tracker add/delete calls shall be logged (api_call_logs).	routes/api_log.py
FR-27	Managers shall view, filter and export entries in Tracker Report within their data scope.	tracker.py /view, /view_daily

4.5 Quality check (QC)

ID	Requirement	Source in code
FR-28	QA/PM shall open an agent's tracker file in the QC Form; the system shall generate a random sample of ceil(rows × QC% ÷ 100) rows (default 10%) shown in file order.	qc-records.controller.ts
FR-29	QA shall mark errors per sampled row using the AFD linked to the project's category, and may add a missed row.	QCFormPage.jsx, lookup-row
FR-30	The score shall be computed automatically: category = 100 − error points; record = average of categories; any error ≥ 100 points makes the record 0; file = average of	QCFormPage.jsx

	records.	
FR-31	The result shall be set by the score and not overridable: 100 = Regular, 98 to <100 = Correction, <98 = Rework.	QCFormPage.jsx
FR-32	For Correction the agent shall upload a corrected file and QA shall mark it Completed or Correction again; for Rework the agent shall upload a reworked file and QA shall run a new QC on it. Cycles shall be recorded in history tables.	services/qc-workflow.service.ts
FR-33	Each QC save shall email the result to the agent.	mail.controller.ts
FR-34	The system shall provide annotated QC downloads with highlighted errors.	download-annotated
FR-35	Managers shall audit QA work (QA Agent Audit) and view QC Report Overview with filters and Error Report export.	routes/qc_audit.py, qc_history_user_based.py
FR-36	QC shall be flagged late if done more than 24 working hours (48 for night trackers) after submission, excluding weekends and holidays.	utils/qc_sla.py

4.6 QA hours (QAH)

ID	Requirement	Source in code
FR-37	The system shall credit QA hours for each QC based on the task's QA target mode; rework QC hours shall be credited only after the agent returns the file.	routes/qa_tracker.py
FR-38	QA shall add manual Feedback, Training, Reporting and Other entries and delete them within 24 hours.	qa_tracker.py
FR-39	The system shall show daily and monthly QA hours against the expected 9-hour day.	qa_tracker.py /day, /monthly

4.7 Roster (ROS)

ID	Requirement	Source in code
FR-40	PM/AM/Admin shall generate next month's roster only during the last week of the current month, for all or one employee, including holidays, week offs, PreJoin and Left days.	routes/roster.py
FR-41	PM/AM/Team Leader shall create draft change requests (DAY_UPDATE, WEEKOFF_SWAP, LEAVE_ADD/UPDATE/DELETE); only the newest draft per person per day shall be kept.	utils/roster_workflow.py
FR-42	Submit shall group drafts into one batch, set the roster to Pending Approval and email Admins.	roster_workflow.py /submit, /notify_approval
FR-43	A submitter shall be able to withdraw a submission before review (requests 'Cancelled due to Withdrawal').	/withdraw
FR-44	Reset All / Reset Employee shall regenerate the roster and cancel open requests ('Cancelled due to Regeneration').	/reset_regenerate
FR-45	Roster changes shall be importable from Excel (template, preview, apply-and-submit).	/excel/*
FR-46	Only Admin and Super Admin shall approve or reject requests; approval shall apply changes oldest-first, increment the roster version and save a snapshot; rejection shall record a reason.	can_approve_reject()
FR-47	Admins shall lock/unlock weeks (Mon–Sun) and months; locked periods shall reject all changes.	utils/roster_week_lock.py
FR-48	Only Admin/Super Admin shall maintain the Holiday Master (add, edit, delete, Excel upload).	routes/holiday.py
FR-49	Every roster action shall be recorded in roster_audit_log; a weekly roster email shall be sendable after approval.	roster_audit_log, /week/email
FR-50	Agents and QA shall view their own roster (My Roster) read-only.	RosterManagement / MyRoster

4.8 Reports, goals & KRA (RPT)

ID	Requirement	Source in code
FR-51	All reports shall be scoped by role: org-wide (1–3), assigned/team (4, 5, 7), self (6).	dashboard.py, kra.py
FR-52	Monthly Goal = Monthly Target + Extra Assign Hours; PM/AM shall edit extra hours; edited goals shall not be overwritten by roster regeneration (goal_locked).	user_monthly_tracker.py
FR-53	KRA shall be calculated monthly with weights Productivity 33%, Quality 33%, Schedule adherence 10%, Reporting 14%, Timeliness 10%, and be exportable to Excel.	utils/kra.py, kra_excel.py

FR-54	Managers and QA in scope shall save KRA day notes; Agents shall not.	/kra/save_notes
FR-55	The Task EOD Report shall merge all files for one task and day, remove duplicate rows on important columns and return one Excel; if any file is missing nothing is generated.	/tracker/eod-report/ generate
FR-56	Billable reports (daily/monthly) shall show delivered vs assigned hours; Tenure shall appear in exports only for roles 1–4.	BillableReport.jsx

4.9 Scheduled jobs & emails (JOB)

ID	Requirement	Source in code
FR-57	At 08:00 daily the system shall write each agent's assigned hours to temp_qc from the roster and tenure.	scheduler.py, /qc/assign-daily-hours
FR-58	Days with 0 production and no file shall receive automatic 100% QC (14-day look-back, Training/Sample projects excluded).	utils/qc_auto_score.py
FR-59	The system shall email the billable report, intraday tracker report and previous-day production report to recipients configured per report.	billable_report_autosend.py, send_tracker_report*.py
FR-60	Only Super Admin shall manage report email recipients (To/CC and per-report switches).	routes/report_email.py

4.10 AI evaluation & SOP library (AIS)

ID	Requirement	Source in code
FR-61	Agents shall run AI evaluation / duplicate check on their files using Google Gemini (gemini-2.5-flash), with their own saved key or the server key.	ai-evaluation.controller.ts
FR-62	Agents shall save/delete a personal Gemini key, stored encrypted.	gemini-key.controller.ts
FR-63	The SOP library shall hold topics and items (links or files); Admin, Super Admin, PM and AM shall manage it; all users shall view it.	routes/sop.py

5. Non-functional requirements

ID	Category	Requirement	Status
NFR-01	Security	All API calls should be authenticated with a server-issued token/session; today the API trusts logged_in_user_id from the client.	Gap
NFR-02	Security	Passwords should be stored as one-way hashes (bcrypt); today they are reversibly encrypted and /user/list returns them decrypted.	Gap
NFR-03	Security	Role checks shall be enforced on the server for every write endpoint (task add/update/delete, user_monthly_tracker/delete, api_log_list lack checks).	Gap
NFR-04	Security	CORS should be restricted to the production domain.	Gap
NFR-05	Security	Secrets (.env) shall not be committed; reset links expire (default 5 min).	Met / partly
NFR-06	Performance	Report screens should load within a few seconds for a month of data; DB connection pooling recommended.	TBC
NFR-07	Availability	Services run under pm2 behind Nginx with HTTPS (Let's Encrypt); daily backups recommended.	TBC
NFR-08	Usability	Role-specific navigation; URLs keep the selected tab (?tab=); toast messages for errors.	Met
NFR-09	Auditability	Roster actions, tracker add/delete and QC cycles shall be logged with user and time.	Met
NFR-10	Time zone	Business dates and the 08:00 job shall use IST.	Met
NFR-11	File limits	Uploads limited to 10 MB (Nginx); files stored on Cloudinary.	Met
NFR-12	Maintainability	Unit tests exist for QC auto-score, roster rules and recipients; wider test coverage TBC.	Partly

6. Data model

MySQL database shared by both APIs. Soft delete: tfs_user and sop_* use is_delete (1 = live); most others use is_active (1 = active). Only qc_correction_history and qc_rework_history have real foreign keys to qc_records; all other links are kept by the code.

Area	Entity	Purpose / key fields
People	tfs_user	user_id, role_id, team_id, designation_id, reporting-line ID lists, user_tenure, user_password (encrypted), is_active, is_delete, deactivated_at, joining_date
People	user_role / team / user_designation / user_permission	Roles 1–7; teams; job titles; creation flags
Setup	project / task / project_category / afd / qc_afd	Projects (JSON team/QA lists, files, SOP links, flags); tasks (target, QC %, important columns, QA target mode); category→AFD; error points
Production	task_work_tracker	tracker_id, user/project/task, date_time, shift, production, targets, billable_hours, tracker_file, qc_status, is_active
Production	tracker_records / temp_qc / api_call_logs	Row hashes (duplicate check); per-agent-day assigned hours & QC; API log
Goals	user_monthly_tracker / project_monthly_tracker	Monthly target, extra hours, working days, goal_locked
QC	qc_records	One per tracker: qa_user_id, agent_id, qc_score, error_list (JSON), status regular/correction/rework, files, counts
QC	qc_correction_history / qc_rework_history / qc_audit	Correction & rework cycles (status correction|rework → completed); manager audits
QA	qa_work_tracker	QA hours per activity with target snapshot
Roster	roster_month	Per person-month: status Draft / Pending Approval / Approved / Locked, version, target days & hours
Roster	roster_day / roster_leave	Day type Working/WeekOff/Holiday/Leave/PreJoin/Left, shift, Full/Half, hours; leaves
Roster	roster_change_request	change_type, payload, batch_id (null = draft), status Pending/Approved/Rejected/Cancelled due to Regeneration/Withdrawal
Roster	roster_version_snapshot / roster_audit_log / roster_week_lock / org_holiday	Approved snapshots, audit trail, week locks, holidays
Other	kra_day_note, sop_topic, sop_item, report_email_recipient	KRA notes; SOP library; report recipients (kra_day_override, kra_month_input unused)

6.1 Status / state models

Entity	States and transitions
roster_month	Draft →(Submit)→ Pending Approval →(Approve)→ Approved →(Lock)→ Locked →(Unlock)→ previous state/Draft; Pending →(Withdraw / all rejected)→ Draft; Approved →(new changes submitted)→ Pending Approval
roster_change_request	Draft (Pending, no batch) →(Submit)→ Pending (batch) → Approved | Rejected | Cancelled due to Withdrawal | Cancelled due to Regeneration; superseded drafts are cancelled
task_work_tracker.qc_status	0 = not QC'd → QC done (record in qc_records); is_active 1 → 0 on delete
qc_records.status	regular | correction | rework (set by score); correction/rework cycles: correction|rework → completed
tfs_user	Active (is_active=1) → Deactivated (is_active=0 + deactivated_at) ; Live (is_delete=1) → Deleted (0)

7. External interfaces and integrations

Interface	Used for	Notes
User interface	React web app	Routes: /login, /reset-password, /dashboard?tab=..., /agent, /agent-projects, /ai-evaluation, /entry, /qc-form

Flask REST API	Core business API	JSON {status, message, data}; mostly POST; prefix /python in production
Node QC REST API	QC & AI	Prefix /api/v1; JSON limit 50 MB
MySQL 8	Persistence	Shared by both APIs
Cloudinary	File storage	Tracker, QC sample, project and SOP files
SMTP (nodemailer / Python)	Email	Password reset, QC results, roster approvals & weekly roster, report emails
Google Gemini	AI evaluation	gemini-2.5-flash via Vercel AI SDK; per-user or server key
Cron + APScheduler	Automation	08:00 assigned-hours job; report email scripts

8. Business rules

ID	Rule
BR-01	Access depends on role, not designation; visibility follows reporting lines.
BR-02	Team Leaders are read-only except roster edit/submit.
BR-03	Billable hours = production ÷ (task target × tenure, tenure counted as at least 1).
BR-04	Assigned hours: full day 9 × tenure (max 9), half day 4.5 × tenure, other day types 0.
BR-05	Night-shift entries before 09:00 belong to the previous day.
BR-06	QC result bands: 100 Regular; 98–<100 Correction; <98 Rework; QA cannot override.
BR-07	Any error worth ≥100 points makes that sampled record 0.
BR-08	QC SLA: 24 working hours (48 night), excluding weekends and holidays.
BR-09	Roster can be generated only in the last week of the month; only Admin/Super Admin approve.
BR-10	Locked weeks cannot be changed by anyone, admins included, until unlocked.
BR-11	Bulk approval runs oldest-first so the latest change for a day wins.
BR-12	Monthly Goal = Monthly Target + Extra Hours; edited goals are preserved on regeneration.
BR-13	KRA weights: 33/33/10/14/10; half-day and unrostered leave count as half a working day.
BR-14	Each task uses exactly one QA target mode; expected QA day = 9 h.
BR-15	Report emails only go to recipients with that report switched on (Super Admin manages).

9. Assumptions, constraints and dependencies

- Assumption: one organisation/tenant; all users in IST.
- Assumption: users have a company email for password reset and QC emails.
- Dependency: Cloudinary, SMTP mailbox and (optionally) Gemini API must be available.
- Constraint: the 08:00 scheduler starts only when app.py is run directly (python app.py); if Flask is run under gunicorn the job would not start – To be confirmed.
- Constraint: the database schema has few foreign keys; integrity relies on application code.
- To be confirmed: exact cron times of report emails (check crontab on server).
- To be confirmed: backend enforcement of salary/incentive visibility (frontend rule only found).
- To be confirmed: data retention and backup policy.
- To be confirmed: target performance/load figures.

10. Known gaps and issues found in the code

#	Finding	Impact	Suggested action
1	No auth token; API trusts logged_in_user_id from the request	Anyone who can reach the API can act as any user	Introduce JWT/session auth
2	Passwords reversibly encrypted; plain-text fallback on login; /user/list returns decrypted passwords	Passwords readable	bcrypt hashing; stop returning passwords
3	Encrypted password strings committed in 'table changes List.txt'; credentials register and test logins files exist in repo docs folders	Credential exposure risk	Remove from repo, rotate
4	If ENCRYPTION_KEY is missing/invalid,	Logs leak key; all logins fail	Fail fast instead

	security.py generates and prints a new key		
5	CORS '*' on both APIs	Cross-site calls allowed	Restrict origins
6	No role check on /task/add|update|delete, /user_monthly_tracker/delete, /api_log_list/logs; Node /gemini-key/get returns decrypted key for any user_id	Unauthorised changes / key exposure	Add server-side checks
7	Node reads ENCRYPTION_SECRET but .env has ENCRYPTION_KEY → Gemini keys use built-in default secret	Weak key protection	Set secret + re-save keys
8	scheduler.py calls /qc/assign-daily-hours without CRON_SECRET header; scheduler only starts under 'python app.py'	08:00 job may silently not run	Send header; run scheduler independently
9	Dead frontend calls in services/qcService.js to Node routes that do not exist	Confusion / future bugs	Remove dead code
10	Task EOD list not scoped by role	AM/QA see other teams' tasks	Scope by role if required
11	New DB connection per request; no pool	Slow under load	Connection pooling
12	Removed project files remain in Cloudinary; kra_day_override / kra_month_input unused	Storage / clutter	Clean up
13	Migrations folder in the copy is empty; schema partly created by code at runtime	Hard to rebuild DB	Consolidate migrations

11. Requirements traceability matrix

Req ID	Module	Workflow chart	Code reference	Test idea
FR-01	Authentication & access	A	routes/auth.py	Verify: The system shall authenticate users by email and password an…
FR-02	Authentication & access	A	utils/user_status.py	Verify: The system shall block login for inactive accounts (is_activ…
FR-03	Authentication & access	A	auth.py, AuthContext.jsx	Verify: On success the system shall return the user profile, role an…
FR-04	Authentication & access	A	routes/AppRoutes.jsx, paths.js	Verify: The website shall route Agents to /agent and all other roles…
FR-05	Authentication & access	A	AuthContext.jsx	Verify: A new login or logout in one browser tab shall log out other…
FR-06	Authentication & access	A	routes/password_reset.py	Verify: Users shall be able to request a password-reset link by emai…
FR-07	Authentication & access	A	password_reset.py	Verify: The system shall verify the reset token before accepting a n…
FR-08	Users & permissions	B	auth.py (registration)	Verify: Only Super Admin, Admin, Project Manager or users with the U…
FR-09	Users & permissions	B	auth.py, utils/validators.py	Verify: User creation shall require name, email, password and role,…
FR-10	Users & permissions	B	tfs_user	Verify: Each user shall have role, team, designation, tenure, joinin…
FR-11	Users & permissions	B	routes/user.py	Verify: Authorised users shall edit users, change passwords and deac…
FR-12	Users & permissions	B	utils/user_status.py	Verify: Deactivated users shall not log in but may add/edit tracker…
FR-13	Users & permissions	B	routes/user.py	Verify: User delete shall be a soft delete (is_delete = 0).…
FR-14	Users & permissions	B	routes/user_permission.py	Verify: Super Admin shall set per-user User Creation and Project Cre…
FR-15	Projects, tasks & AFD	C	routes/afd_master.py, qc_afd.py	Verify: Authorised users shall create, edit and delete AFD masters w…
FR-16	Projects, tasks & AFD	C	routes/project_category.py	Verify: Project categories shall be created and linked to exactly on…

FR-17	Projects, tasks & AFD	C	routes/project.py	Verify: Projects shall store code, category, manager, teams, QA list…
FR-18	Projects, tasks & AFD	C	routes/task.py	Verify: Tasks shall store target per hour, QC %, important columns a…
FR-19	Projects, tasks & AFD	C	project.py, AuthContext.jsx	Verify: Only users permitted to manage projects (roles 1–4 or Projec…
FR-20	Daily tracker	F	routes/tracker.py	Verify: Agents shall add tracker entries with project, task, shift (…
FR-21	Daily tracker	F	tracker.py	Verify: The system shall reject tracker entries for future dates….
FR-22	Daily tracker	F	tracker.py	Verify: A NIGHT-shift entry created before 09:00 shall be dated to t…
FR-23	Daily tracker	F	tracker.py	Verify: The system shall compute tenure_target = task_target × tenur…
FR-24	Daily tracker	F	qc-backend tracker-process.controller.ts	Verify: When a project has duplicate check enabled, uploaded Excel r…
FR-25	Daily tracker	F	tracker.py	Verify: Agents shall delete only same-day entries (night entries unt…
FR-26	Daily tracker	F	routes/api_log.py	Verify: Tracker add/delete calls shall be logged (api_call_logs)….
FR-27	Daily tracker	F	tracker.py /view, /view_daily	Verify: Managers shall view, filter and export entries in Tracker Re…
FR-28	Quality check	G	qc-records.controller.ts	Verify: QA/PM shall open an agent's tracker file in the QC Form; the…
FR-29	Quality check	G	QCFormPage.jsx, lookup-row	Verify: QA shall mark errors per sampled row using the AFD linked to…
FR-30	Quality check	G	QCFormPage.jsx	Verify: The score shall be computed automatically: category = 100 −…
FR-31	Quality check	G	QCFormPage.jsx	Verify: The result shall be set by the score and not overridable: 10…
FR-32	Quality check	G	services/qc-workflow.service.ts	Verify: For Correction the agent shall upload a corrected file and Q…
FR-33	Quality check	G	mail.controller.ts	Verify: Each QC save shall email the result to the agent….
FR-34	Quality check	G	download-annotated	Verify: The system shall provide annotated QC downloads with highlig…
FR-35	Quality check	G	routes/qc_audit.py, qc_history_user_based.py	Verify: Managers shall audit QA work (QA Agent Audit) and view QC Re…
FR-36	Quality check	G	utils/qc_sla.py	Verify: QC shall be flagged late if done more than 24 working hours…
FR-37	QA hours	H	routes/qa_tracker.py	Verify: The system shall credit QA hours for each QC based on the ta…
FR-38	QA hours	H	qa_tracker.py	Verify: QA shall add manual Feedback, Training, Reporting and Other…
FR-39	QA hours	H	qa_tracker.py /day, /monthly	Verify: The system shall show daily and monthly QA hours against the…
FR-40	Roster	D	routes/roster.py	Verify: PM/AM/Admin shall generate next month's roster only during t…
FR-41	Roster	D	utils/roster_workflow.py	Verify: PM/AM/Team Leader shall create draft change requests (DAY_UP…
FR-42	Roster	D	roster_workflow.py /submit, /notify_approval	Verify: Submit shall group drafts into one batch, set the roster to…
FR-43	Roster	D	/withdraw	Verify: A submitter shall be able to withdraw a submission before re…
FR-44	Roster	D	/reset_regenerate	Verify: Reset All / Reset Employee shall regenerate the roster and c…
FR-45	Roster	D	/excel/*	Verify: Roster changes shall be importable from Excel (template, pre…
FR-46	Roster	D	can_approve_reject()	Verify: Only Admin and Super Admin shall approve or reject requests;…
FR-47	Roster	D	utils/roster_week_lock.py	Verify: Admins shall lock/unlock weeks (Mon–Sun) and months; locked…

FR-48	Roster	D	routes/holiday.py	Verify: Only Admin/Super Admin shall maintain the Holiday Master (ad…
FR-49	Roster	D	roster_audit_log, /week/email	Verify: Every roster action shall be recorded in roster_audit_log; a…
FR-50	Roster	D	RosterManagement / MyRoster	Verify: Agents and QA shall view their own roster (My Roster) read-o…
FR-51	Reports, goals & KRA	J	dashboard.py, kra.py	Verify: All reports shall be scoped by role: org-wide (1–3), assigne…
FR-52	Reports, goals & KRA	J	user_monthly_tracker.py	Verify: Monthly Goal = Monthly Target + Extra Assign Hours; PM/AM sh…
FR-53	Reports, goals & KRA	J	utils/kra.py, kra_excel.py	Verify: KRA shall be calculated monthly with weights Productivity 33…
FR-54	Reports, goals & KRA	J	/kra/save_notes	Verify: Managers and QA in scope shall save KRA day notes; Agents sh…
FR-55	Reports, goals & KRA	J	/tracker/eod-report/generate	Verify: The Task EOD Report shall merge all files for one task and d…
FR-56	Reports, goals & KRA	J	BillableReport.jsx	Verify: Billable reports (daily/monthly) shall show delivered vs ass…
FR-57	Scheduled jobs & emails	I	scheduler.py, /qc/assign-daily-hours	Verify: At 08:00 daily the system shall write each agent's assigned…
FR-58	Scheduled jobs & emails	I	utils/qc_auto_score.py	Verify: Days with 0 production and no file shall receive automatic 1…
FR-59	Scheduled jobs & emails	I	billable_report_autosend.py, send_tracker_report*.py	Verify: The system shall email the billable report, intraday tracker…
FR-60	Scheduled jobs & emails	I	routes/report_email.py	Verify: Only Super Admin shall manage report email recipients (To/CC…
FR-61	AI evaluation & SOP library	K	ai-evaluation.controller.ts	Verify: Agents shall run AI evaluation / duplicate check on their fi…
FR-62	AI evaluation & SOP library	K	gemini-key.controller.ts	Verify: Agents shall save/delete a personal Gemini key, stored encry…
FR-63	AI evaluation & SOP library	K	routes/sop.py	Verify: The SOP library shall hold topics and items (links or files)…

Appendix A. Workflow diagrams

Full-page versions with explanations are in the companion 'HRMS Detailed System Workflow' document.

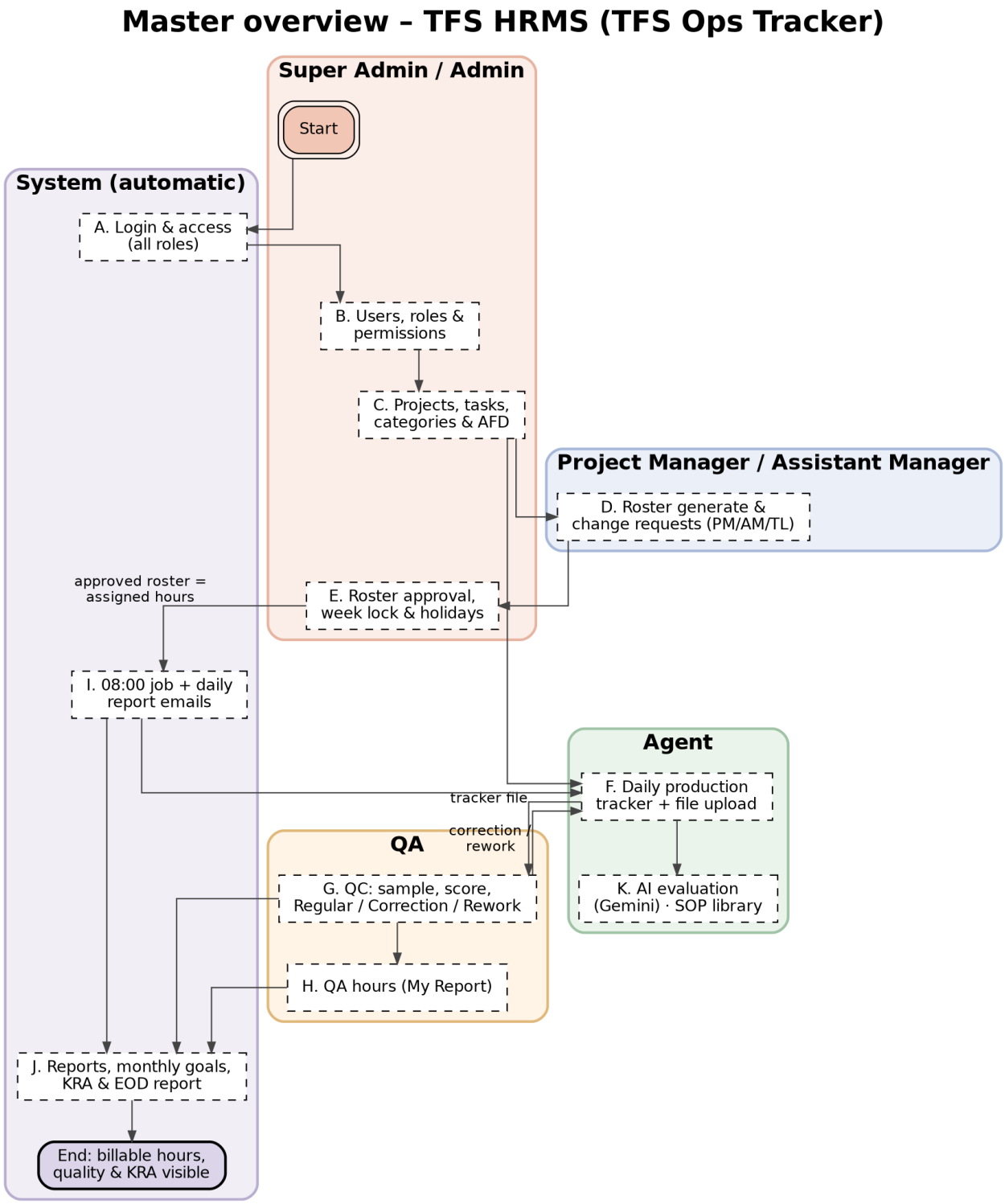


Figure – Master overview

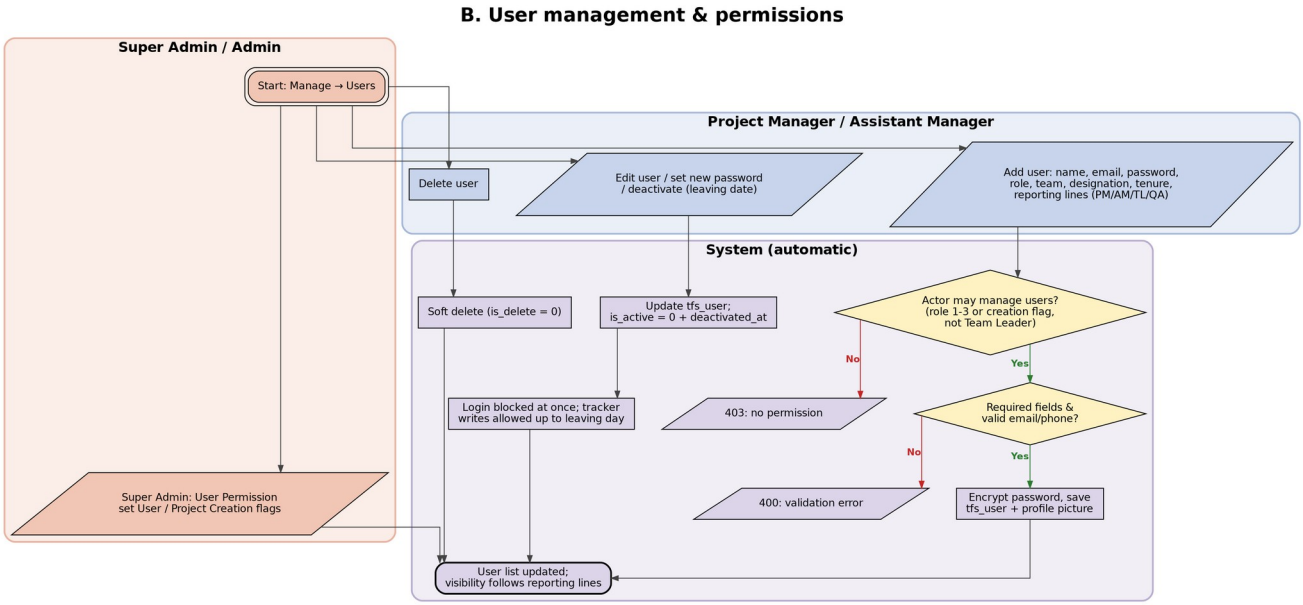


Figure – B. User management & permissions

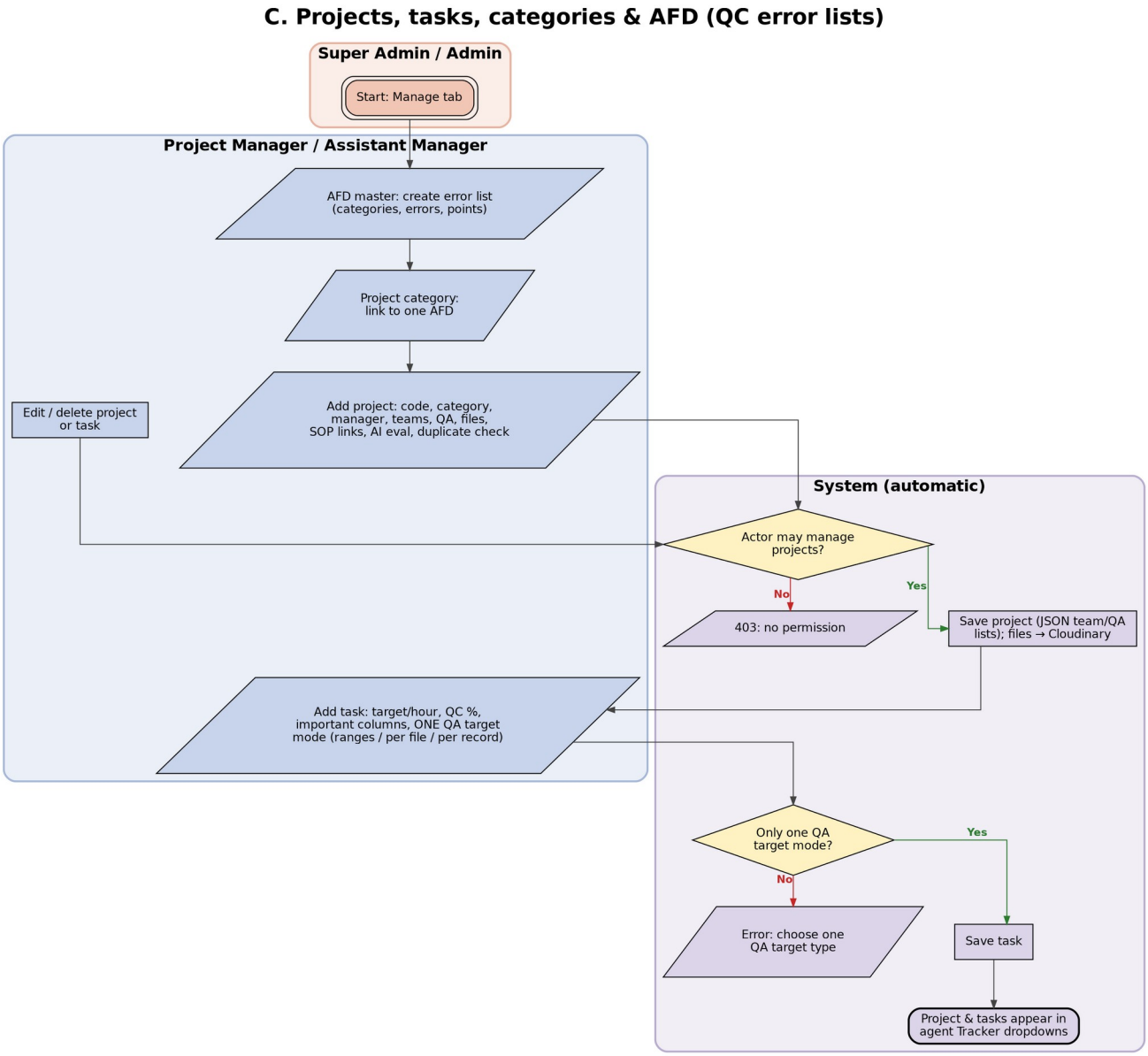


Figure – C. Projects, tasks, categories & AFD

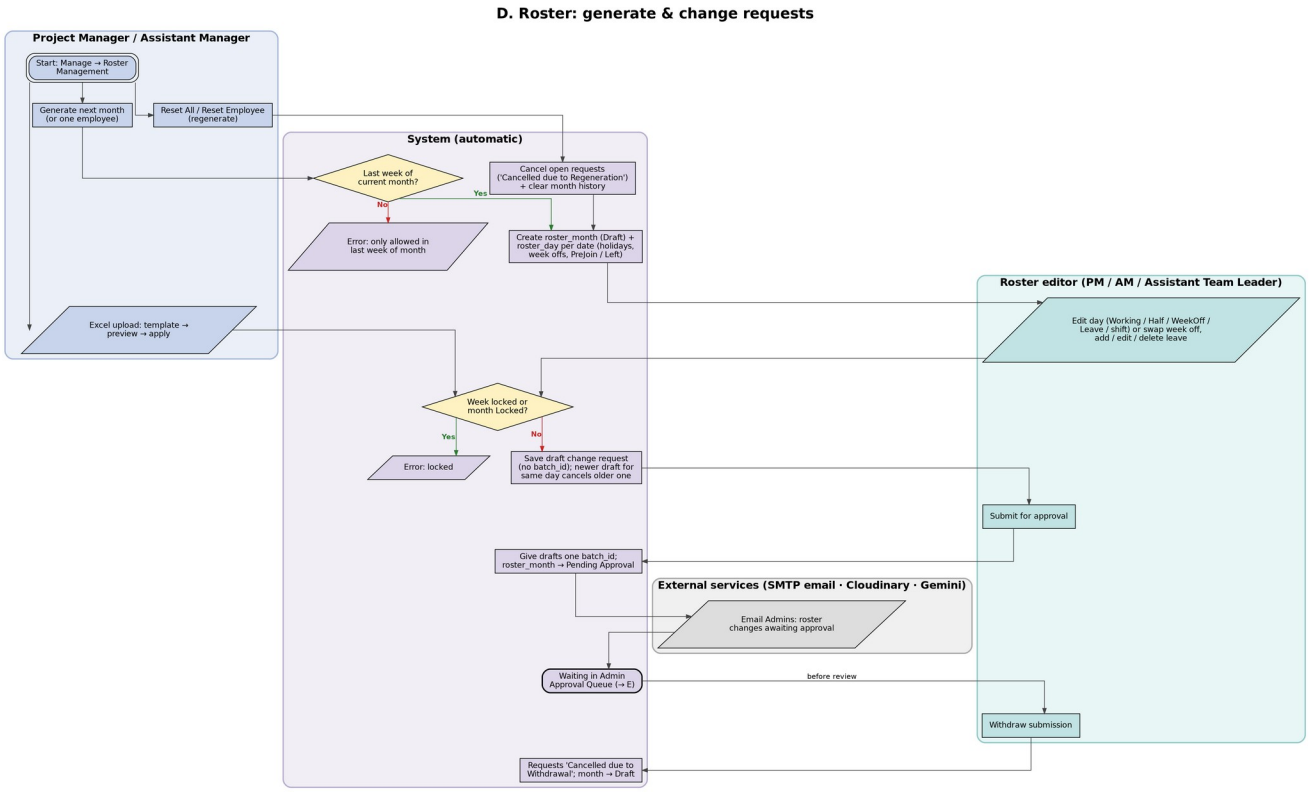


Figure – D. Roster: generate & change requests

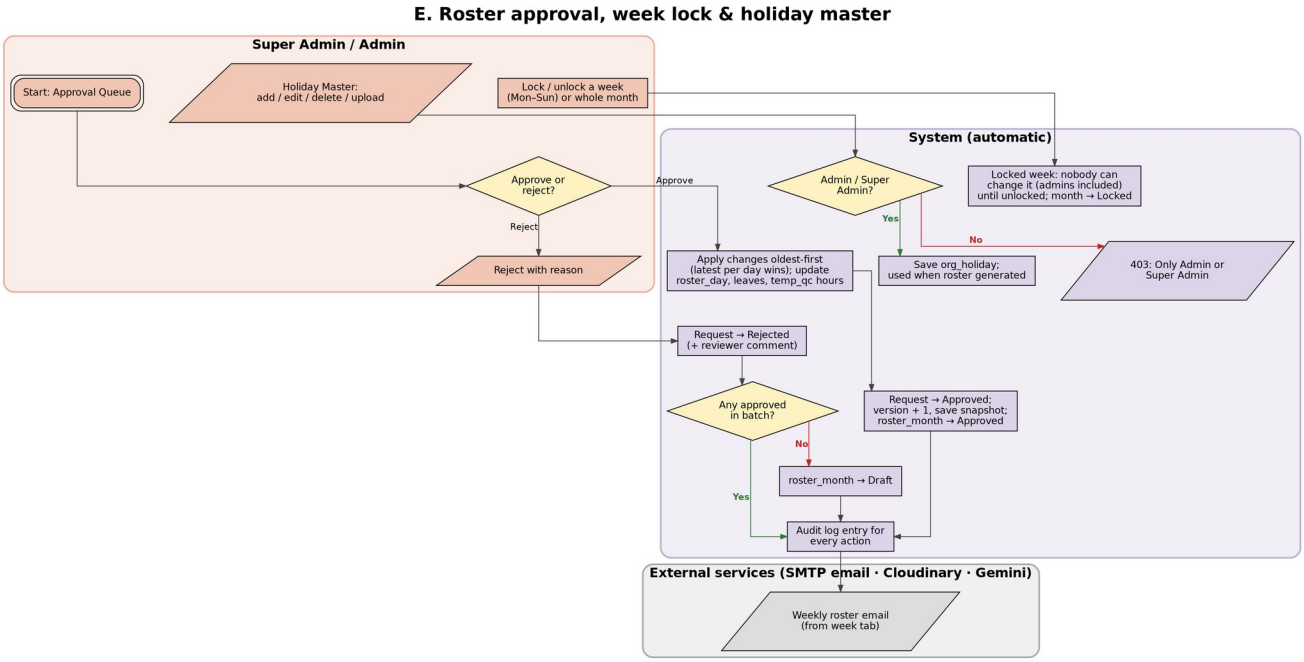


Figure – E. Roster approval, week lock & holidays

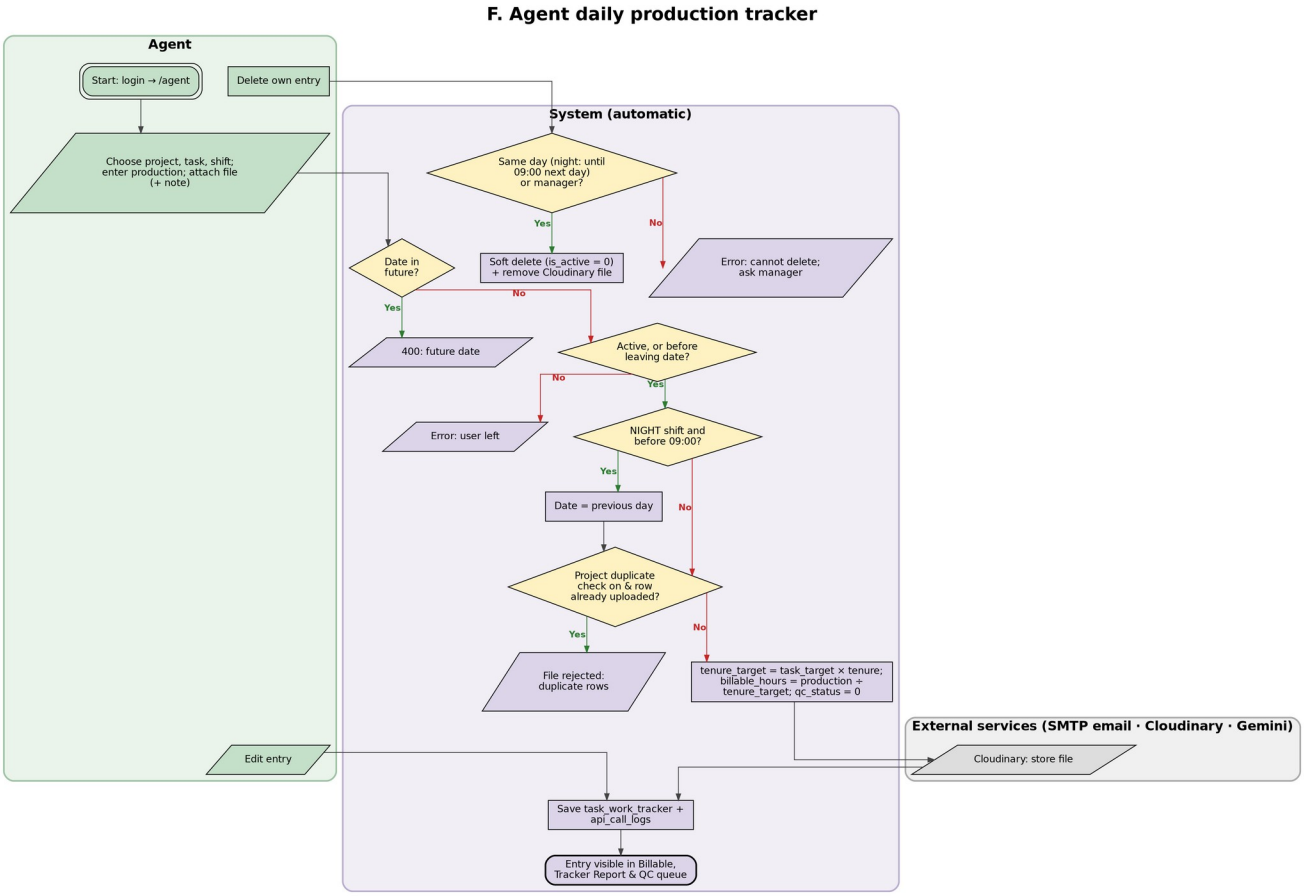


Figure – F. Agent daily production tracker

G. QC process: sample → score → Regular / Correction / Rework

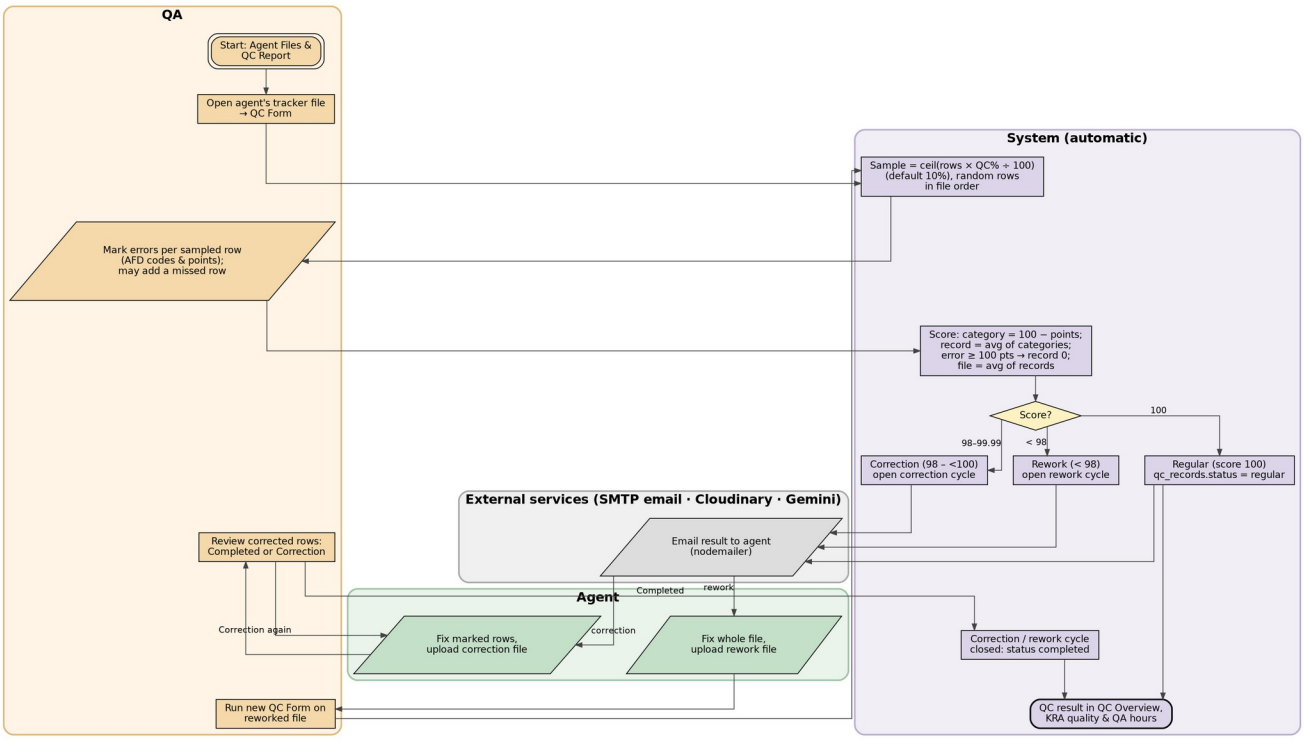


Figure – G. QC process

H. QA hours (My Report)

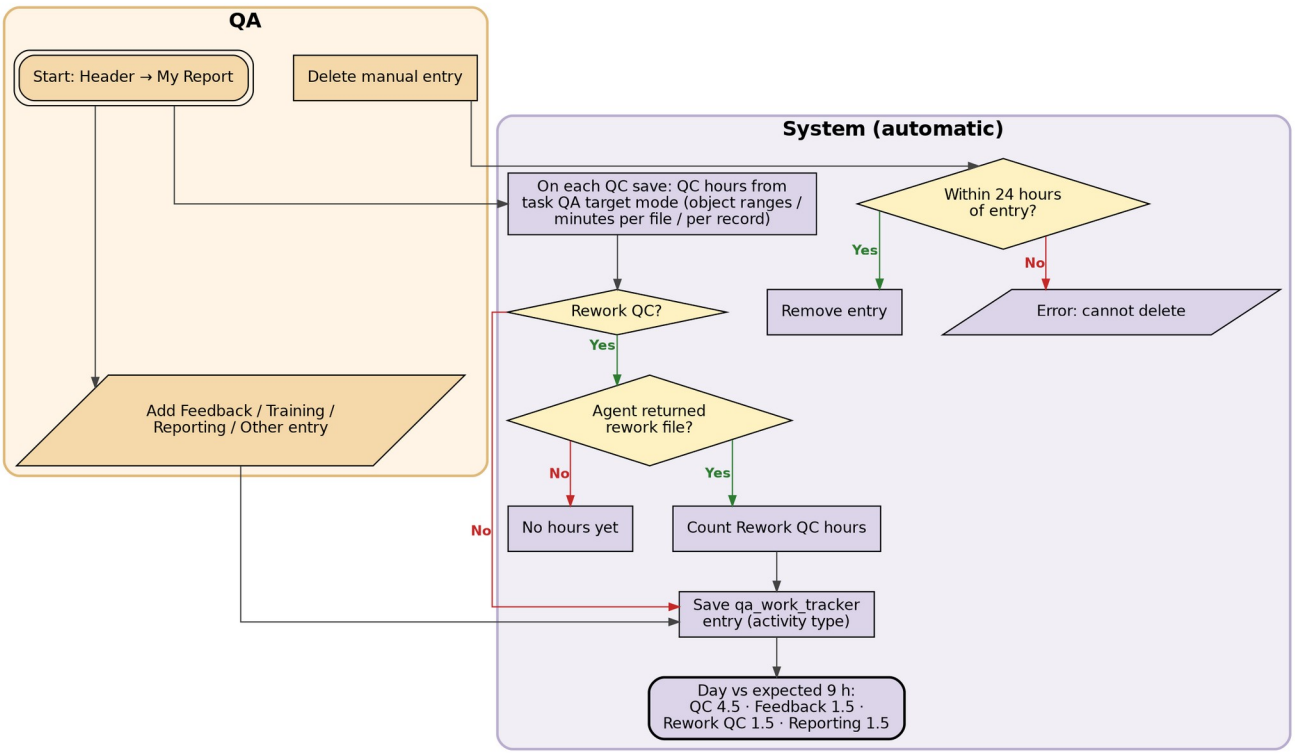


Figure – H. QA hours (My Report)

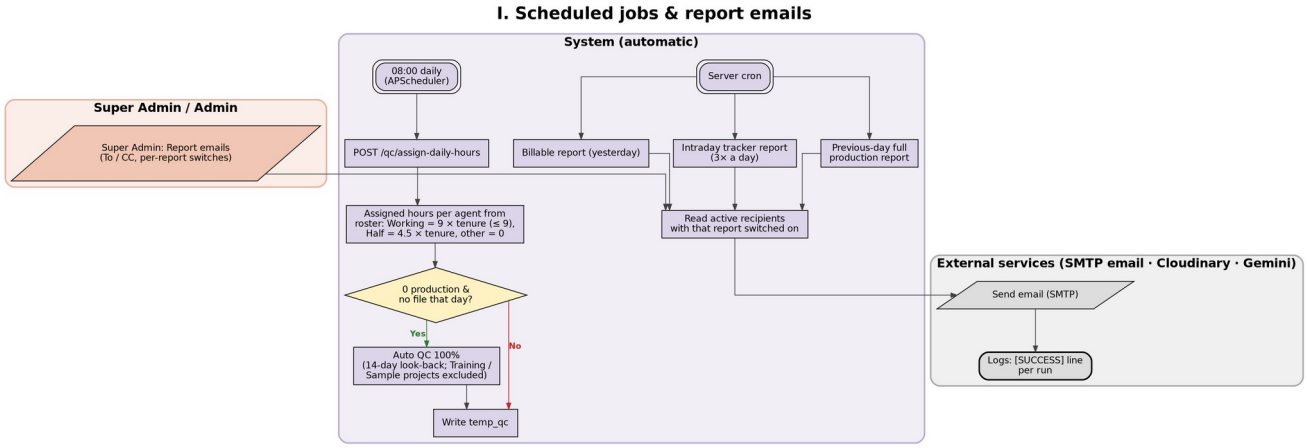


Figure – I. Scheduled jobs & report emails

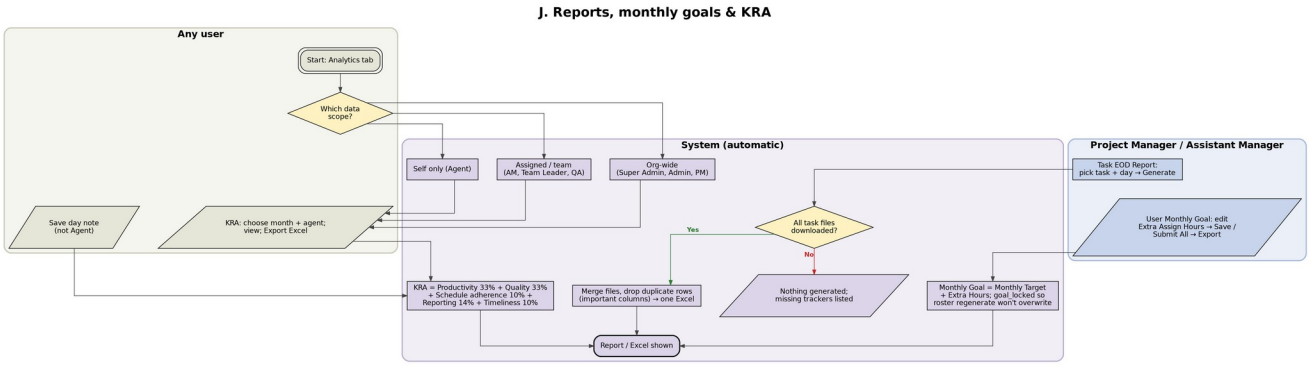


Figure – J. Reports, monthly goals & KRA

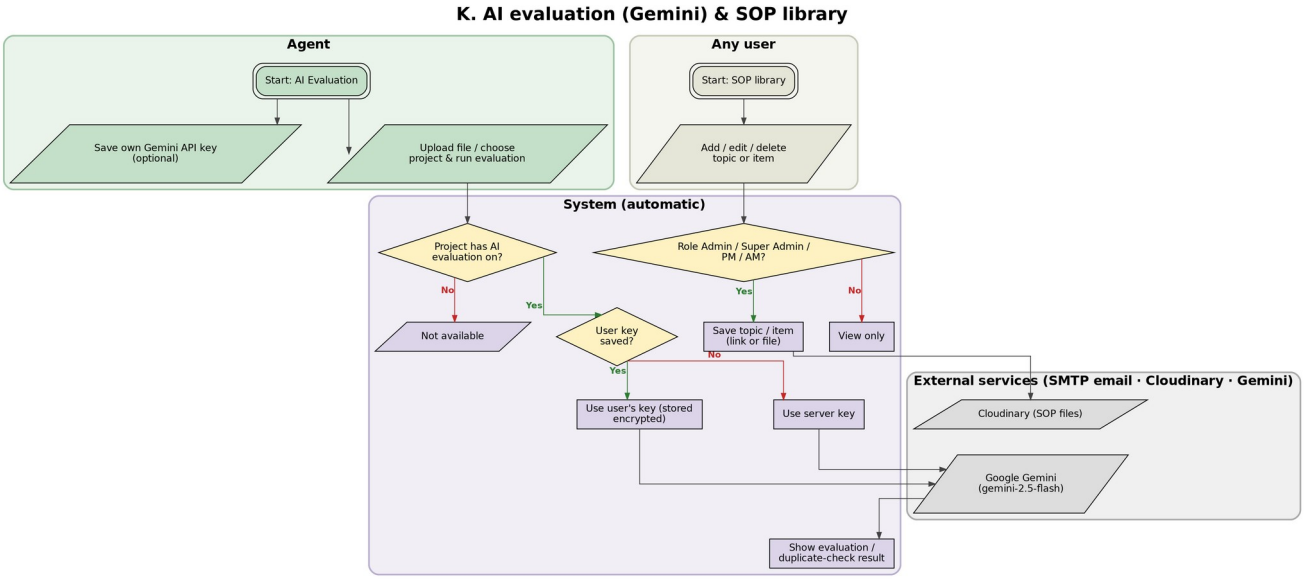


Figure – K. AI evaluation & SOP library